

Simulation study for debiased learning

Simulation setup

Let A be a sample of size $n = 500$ selected from the population U of size $N = 5000$. Let y be a variable of interest and we try to estimate the population total $t_y = \sum_{i \in U} y_i$ and $\mathbf{x}_i$ of size p be the auxiliary variables associated with y where $x_{ij} \stackrel{i.i.d.}{\sim} N(2, 1)$. Suppose the following superpopulation model holds

$$y_i = \mathbf{x}_i^T \boldsymbol{\beta} + \varepsilon_i, \quad \varepsilon_i \stackrel{i.i.d.}{\sim} N(0, \sigma^2), i = 1, \dots, N$$

where $\sigma^2 = 1$ and $\boldsymbol{\beta} = (1, \dots, 1, 0, \dots, 0)$ with $s = 5$ 1s and $p - s = 195$ 0s. We assume the samples are drawn from the SRS sampling.

We consider the following estimators.

- Difference estimator

$$\hat{t}_y = \left(\sum_{i \in U} \mathbf{x}_i \right)^T \boldsymbol{\beta} + \sum_{i \in A} d_i (y_i - \mathbf{x}_i^T \boldsymbol{\beta})$$

- GREG estimator

$$\hat{t}_y = \left(\sum_{i \in U} \mathbf{x}_i \right)^T \hat{\boldsymbol{\beta}} + \sum_{i \in A} d_i (y_i - \mathbf{x}_i^T \hat{\boldsymbol{\beta}})$$

where $\hat{\boldsymbol{\beta}} = (\sum_{i \in A} d_i \mathbf{x}_i \mathbf{x}_i^T)^{-1} (\sum_{i \in A} d_i \mathbf{x}_i y_i)$

- Model-based estimator

$$\hat{t}_y = \left(\sum_{i \in U} \mathbf{x}_i \right)^T \hat{\boldsymbol{\beta}}$$

- Debiased GREG estimator(Proposed)

$$\hat{t}_y = \left(\sum_{i \in U} \mathbf{x}_i \right)^T \left(\frac{\sum_{i \in A_2} d_i}{\sum_{i \in A} d_i} \hat{\boldsymbol{\beta}}_1 + \frac{\sum_{i \in A_1} d_i}{\sum_{i \in A} d_i} \hat{\boldsymbol{\beta}}_2 \right) + \sum_{i \in A_1} d_i (y_i - \mathbf{x}_i^T \hat{\boldsymbol{\beta}}_2) + \sum_{i \in A_2} d_i (y_i - \mathbf{x}_i^T \hat{\boldsymbol{\beta}}_1)$$

where A_1 and A_2 are the subsamples of A with $A_1 \cup A_2 = A$, and $\hat{\boldsymbol{\beta}}_1$ and $\hat{\boldsymbol{\beta}}_2$ are the regression coefficients built upon the sample A_1 and sample A_2 respectively.

- Design unbiased GREG estimator(Sande, 2021)

$$\hat{t}_y = \left(\sum_{i \in U} \mathbf{x}_i \right)^T \hat{\boldsymbol{\beta}}_1 + \sum_{i \in A_1} (y_i - \mathbf{x}_i^T \hat{\boldsymbol{\beta}}_1) + \sum_{i \in A_2} \frac{1}{P(i \in A_2 | i \in A)} (y_i - \mathbf{x}_i^T \hat{\boldsymbol{\beta}}_1)$$

```
res = NULL
res2 = NULL
res3 = NULL
SIMNUM = 500
for(simnum in 1:SIMNUM){
```

```

set.seed(simnum)
Index = sample(1:N, size = n, replace = FALSE, prob = pi1)
y_s = y[Index]
X_s = X[Index, ]
d1_s = d1[Index]
lm_obj = lm(y_s ~ X_s, weights = d1_s)
beta_hat = lm_obj$coefficients[-1]
#drop(solve(t(X_s) %*% diag(d1_s) %*% X_s, t(X_s) %*% diag(d1_s) %*% y_s))

# beta_hat = unname(lm(y_s ~ t(apply(X_s, 1, function(k) k
# - colSums(X_s * d1_s) / N)), weights = d1_s)$coefficients[-1])

y_HT = sum(y_s * d1_s)

y_diff = drop(colSums(X) %*% beta + sum((y_s - drop(X_s %*% beta)) * d1_s))

y_GREG = drop(colSums(X) %*% beta_hat + sum((y_s - drop(X_s %*% beta_hat)) * d1_s))

y_model = drop(colSums(X) %*% beta_hat)

Index_sub1 = Index[sample(1:n, size = m, replace = FALSE)]
Index_sub2 = Index[!(Index %in% Index_sub1)]

y_s1 = y[Index_sub1]
X_s1 = X[Index_sub1, ]
X1_s1 = cbind(1, X)[Index_sub1, ]
d1_s1 = d1[Index_sub1]
lm_obj1 = lm(y_s1 ~ X_s1, weights = d1_s1)
beta_hat1 = lm_obj1$coefficients[-1]

y_s2 = y[Index_sub2]
X_s2 = X[Index_sub2, ]
d1_s2 = d1[Index_sub2]
lm_obj2 = lm(y_s2 ~ X_s2, weights = d1_s2)
beta_hat2 = lm_obj2$coefficients[-1]

# y_debiased = drop(colSums(X) %*% beta_hat + sum((y_s1 - drop(X_s1 %*% beta_hat2)) * d1_s1)
# + sum((y_s2 - drop(X_s2 %*% beta_hat1)) * d1_s2))

y_debiased = drop(colSums(X) %*% (sum(d1_s2) / sum(d1_s) * beta_hat1 + sum(d1_s1) / sum(d1_s) * beta_hat2) +
sum((y_s1 - drop(X_s1 %*% beta_hat2)) * d1_s1) + sum((y_s2 - drop(X_s2 %*% beta_hat1)) * d1_s2))

y_unbiased = drop(colSums(X) %*% beta_hat1 + sum((y_s1 - drop(X_s1 %*% beta_hat1)))
+ sum((y_s2 - drop(X_s2 %*% beta_hat1)) * 2))

Omega = matrix(1 - n / (n-1) * (N-1) / N, nr = n, nc = n)
diag(Omega) = 1 - n / N

e_s = y_s - X_s %*% beta
sigma_diff = sqrt(drop(t(e_s) %*% Omega %*% e_s * N^2 / n^2))

# sum(sapply(1:n, function(k) sapply(1:n, function(l)

```

```

#   ifelse(k == l, 1 - n / N, 1 - n / (n - 1) * (N - 1) / N) * e_s[k] * e_s[l] * N / n * N / n ))) / N

e_s = y_s - X_s %>% beta_hat
sigma_GREG = sqrt(drop(t(e_s) %>% Omega %>% e_s * N^2 / n^2))

# e_s = ifelse(Index %in% Index_sub1, y_s - X_s %>% beta_hat2, y_s - X_s %>% beta_hat1)
# # e_s = ifelse(Index %in% Index_sub1, y_s - X_s %>% beta_hat1, y_s - X_s %>% beta_hat2)
# # e_s = c(y_s1 - X_s1 %>% beta_hat2, y_s2 - X_s2 %>% beta_hat1)
# sigma_Debaised = sqrt(drop(t(e_s) %>% Omega %>% e_s * N^2 / n^2))

e_s1 = ifelse(Index %in% Index_sub1, y_s - X_s %>% beta_hat1, 0)
e_s2 = ifelse(Index %in% Index_sub2, y_s - X_s %>% beta_hat2, 0)

sigma_Debaised = sqrt(drop((t(e_s1) %>% Omega %>% e_s1 + t(e_s2) %>% Omega %>% e_s2) * N^2 / n^2))

res = rbind(res, c(Diff = y_diff, GREG = y_GREG, Model = y_model, Debaised = y_debaised, Unbiased = y_unbiased))

res2 = rbind(res2, c(Diff = sigma_diff, GREG = sigma_GREG, Debaised = sigma_Debaised))

# res3 = rbind(res3, c(sum((y_s1 - drop(X_s1 %>% beta_hat2)) * d1_s1), sum((y_s2 - drop(X_s2 %>% beta_hat1)) * d2_s2)))

res3 = rbind(res3, c(colMeans(X_s), beta_hat))

}

BIAS = colMeans(res - t_y)
SE = apply(res, 2, function(x) sqrt(var(x) * (length(x)-1)/length(x) ))
RMSE = apply(res - t_y, 2, function(x) sqrt(mean(x^2)))

cbind(BIAS, SE, RMSE)

```

##	BIAS	SE	RMSE
## Diff	22.493403	220.9426	222.0847
## GREG	27.427755	274.4130	275.7803
## Model	-2755.787515	7829.9093	8300.7135
## Debaised	17.064931	502.7975	503.0870
## Unbiased	-3496.489480	16786.8144	17147.0865
## HT	9.820459	543.7638	543.8525

Variance estimation

Consider the following variance estimation method for estimating the variance of debaised estimator:

$$V_p \left(\sum_{i \in A_1} \frac{y_i - \mathbf{x}_i \hat{\beta}_2}{\pi_i} \right) + V_p \left(\sum_{i \in A_2} \frac{y_i - \mathbf{x}_i \hat{\beta}_1}{\pi_i} \right)$$

```

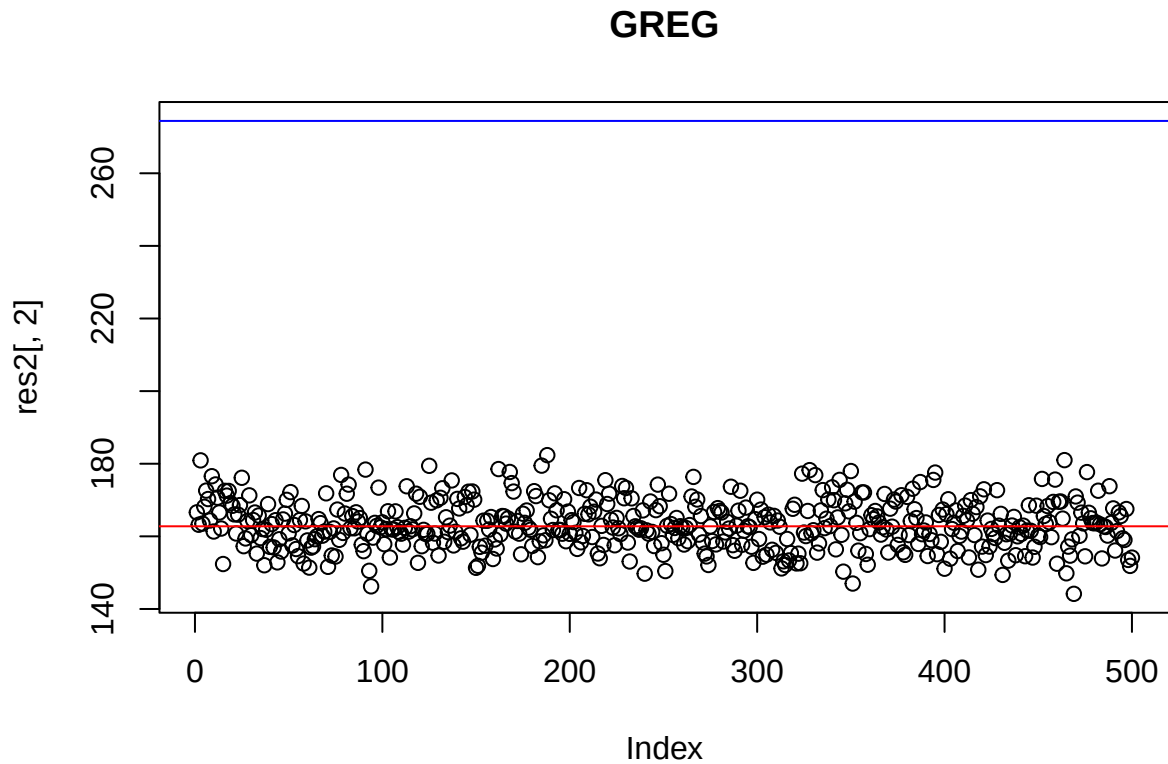
BIAS2 = colMeans(res2 - rep(SE[c(1,2,4)], each = SIMNUM))
SE2 = apply(res2, 2, function(x) sqrt(var(x) * (length(x)-1)/length(x) ))
RMSE2 = apply(res2 - rep(SE[c(1,2,4)], each = SIMNUM), 2, function(x) sqrt(mean(x^2)))
MidErr2 = apply(res2 - rep(SE[c(1,2,4)], each = SIMNUM), 2, median)

cbind(BIAS2, SE2, RMSE2, MidErr2)

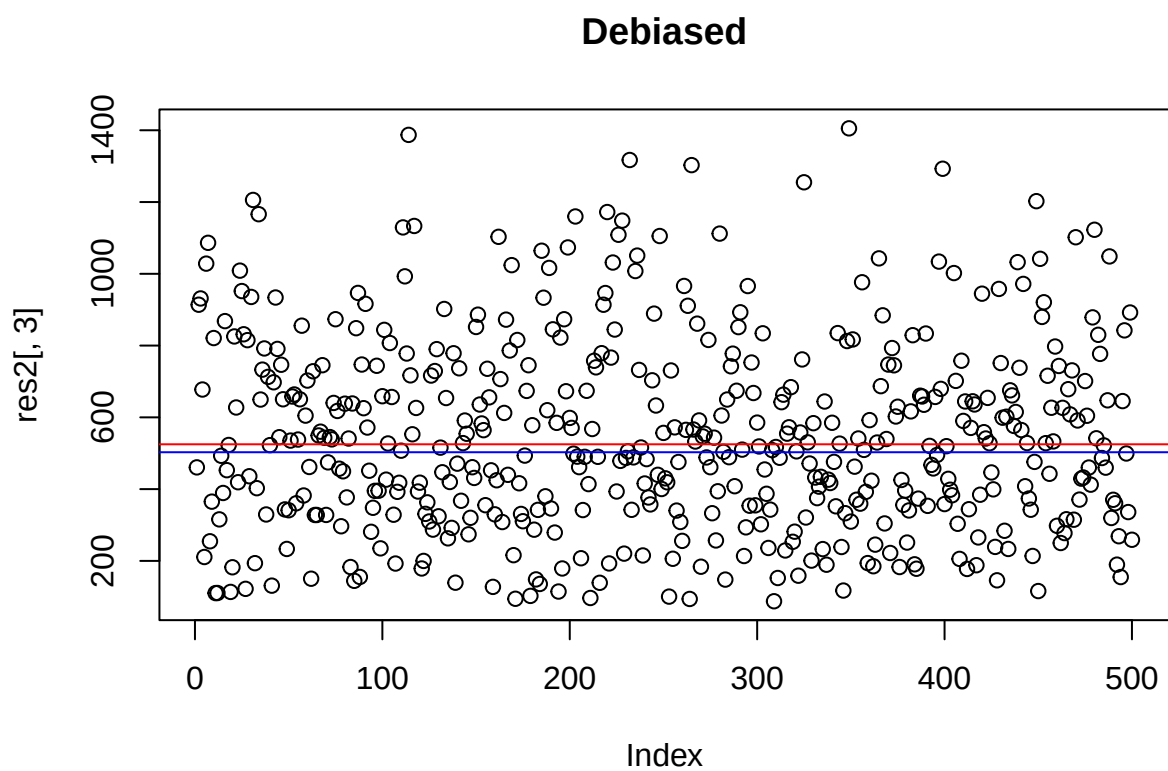
```

```
##          BIAS2      SE2      RMSE2      MidErr2
## Diff      -9.977691    6.62237    11.9754   -10.10267
## GREG      -111.126214   6.64782   111.3249  -111.63912
## Debiased   48.174842  267.71327   272.0133    22.11643
```

```
plot(res2[,2], main = "GREG", ylim = c(min(res2[,2]), SE[2]))
abline(h = SE[2], col = "blue")
abline(h = median(res2[,2]), col = "red")
```



```
plot(res2[,3], main = "Debiased")
abline(h = SE[4], col = "blue")
abline(h = median(res2[,3]), col = "red")
```



```
# median(res2[,3])
```

```
hist(res2[,3], main = "SE of Debiased estimator")
```

SE of Debiased estimator

